Exercise - Floorplanning

This exercise is still being updated for the 2026FS class. You can look at it but don't expect it to be in its final version until this message is displayed.

Overview

Goals of this exercise

- All subsequent steps of the backend design flow, follow the general plan set out during floorplanning. Understand how to go through this step, determine the circuit area and the aspect ratio (height/width) of your design.
- Learn how macro blocks and I/O cells are placed
- Create the power connections
- Experience how decisions in floorplanning affect other parts of the design flow

About the style

We will use a number of different styles to identify different types of actions as shown below:

Student Task

Parts of the text that have a gray background, like the current paragraph, indicate steps required to complete the exercise.

Actions that require you to select a specific menu will be shown like the following: `menu → sub-menu → sub-sub-menu`

Whenever there is an option or a tab that can be found in the current view/menu we will use a `BUTTON` to indicate such an option.

Throughout the exercise you will be asked to enter certain commands using the command-line.

```
sh> command to be entered on the Linux command line
```

Note: *there are many reasons for using a command-line. Some functionality can not not be accessed through GUI commands, and in some cases, using the command-line will be much faster. Most importantly, things you enter on the command-line can be converted into a script and executed repeatedly.*

Preparation

Student Task 1

For ETH users:

- Setup the working directory for this exercise by calling our install script and navigating to the newly created directory. **Important**: make sure you install your exercises on the local hard disks rather than in the accounts home directories (which have a limited disk quota). For this sake, create a directory with your username on the local hard disk (e.g. `/scratch/vlsi2_XXfs26/`), and install the exercise files there. Check out the dedicated Exercise setup page for more information.

```
sh > mkdir -p /scratch/$USER
sh > cd /scratch/$USER
sh > vlsi2 ex06
sh > cd ex06
```

Alternatively, you can directly call the install script for each exercise which you can find under `/home/vlsi2/ex06/install.sh`.

For non-ETH users:

Note: The exercises have been recently updated for the Spring semester 2026. It is possible that some exercise branches on Github require some additional time to be updated accordingly. Please reach out to a maintainer in case of issues.

- Retrieve the files from this exercise from Github:

```
sh > git clone https://github.com/pulp-platform/croc.git
sh > cd croc
sh > git checkout ex06
```

OpenROAD configuration

In this exercise, you have to enter several commands on the OpenRoad terminal.
Take care of saving the commands in a script (`scripts/myFloorplan.tcl`).
If you want to exit the terminal and re-run the commands, you can type on the OpenROAD terminal:

```
OpenROAD > source scripts/myFloorplan.tcl
```

Student Task 2

Let's enter the croc directory and generate the technology files.

```
sh > cd croc
sh > icdesign ihp13 -update all -nogui
```

Let's launch the OpenRoad tool as seen in the previous exercise.

```
sh > cd openroad
sh > oseda -2025.01 openroad -gui
```

The first step is to setup OpenRoad.
First, set some variables and some functions to use in the scripts.

```
OpenROAD > source scripts/startup.tcl
```

Now that the tool has been setup, you need to provide the information about your project.
The information you have to provide are:

- the synthesis netlist, which has been generated at the end of the synthesis and contains the information about your design.
- the technology libraries, to map your design to existing cells and macro.

The script with the technology file information has been already prepared for you (`scripts/init_tech.tcl`)
.

The synthesis netlist has been generated in `croc/yosys/out`.
Look at `https://openroad.readthedocs.io/en/latest/main/src/README.html` to understand how to read the verilog netlist. Then, write the command on the OpenRoad terminal.

```
OpenROAD > <OpenROAD command> <Synthesis Netlist Directory>
```

Finally, you flatten the design and make a database.
The database is needed if you want to save your project (see `https://openroad.readthedocs.io/en/latest/main/src/README.html` for more information).
To flatten the design, you need to specify the top level module of your design.
The top level is `croc_chip`, but you can directly use the variable `top_design` from the startup script.

```
OpenROAD > link_design $top_design
```

Now you are ready for the Floorplanning.

ezelioli -- Add overview of main steps of floorplanning according to script possibly with diagram for quick visual overview

Example of floorplan flow overview:

- Setup / initialization
  - Reading the netlist from synthesis stage
  - Linking the design
  - Reading the constraints
  - Sanity checks
- Floorplan initialization
  - Die and core area definition
  - Core site definition
- Pad ring
  - IO ring definition (make_io_sites)
  - Manual pads placement (place_pad)
- Global nets connection (power nets)
- Tracks creation
- Macros placement (SRAMs)
- TAP cells insertion
  - cells insertion
  - global nets connection
- Power grid
  - Power ring
  - Power rails
  - Power grid
  - Macros power rings and grids
  - Power grid connections

Introduction

In this exercise you will understand:

- how to **size a chip**
- how to **place IO pads**
- how to **place macro cells**
- how to **design the power distribution network**

This process is called floorplanning.

For a standard design, our main concern would be to find a floorplan that will result in the smallest possible area, while fulfilling all performance and reliability requirements. This is purely driven by economical reasons, since chip costs are mainly determined by the area. In some cases there are additional geometrical constraints the manufacturing company may impose. Back-end design is not only used for complete chips. Macro-cells that will be part of a larger system-on-chip design can also be designed in this way. In such cases there might be even more restrictions. For example, certain metal layers might be reserved for the system level.

Before diving into the exercises, let's review some concepts.

The **die area** corresponds to the total silicon area available to place pads and core cells.
All pads (I/O, power and corner) are placed in the **padframe**, which is another level of hierarchy where both the original design and the I/O cells are instantiated.
The remaining area can be used for the core of the chip.
The **core area** is surrounded by a **core power ring** to evenly distribute the power all around the chip and in its simplest form this consists of two wide metal lines (one for VDD, one for VSS). In order to leave room for the power ring, we need to leave a certain **I/O to core spacing**.

The core area is filled with **standard cell rows** on which all **standard cells** will be placed. **Utilization** indicates to what percentage the standard cell rows are filled. The standard cells are designed in such a way that, when placed next to each other their VDD and VSS pins can be connected with one horizontal metal line each. These power connections are relatively narrow and run over the entire width of the core area to connect to the core power rings. This may create problems for designs that consume much current, since the cells towards the middle would not have a good power connection. To improve this, the two uppermost metal layers are usually used to place a power grid over the core area. The vertical and horizontal power stripes of that grid connect down through all metal layers to the narrow standard cell power connections.

In the core area, we will also need to reserve some room for our **macro-cells**. Most macro-cells need some free space around to make signal connections, to add a block power ring, or simply to prevent standard cells from being placed too close to the macro. We will define a **block halo** to specify this free space.

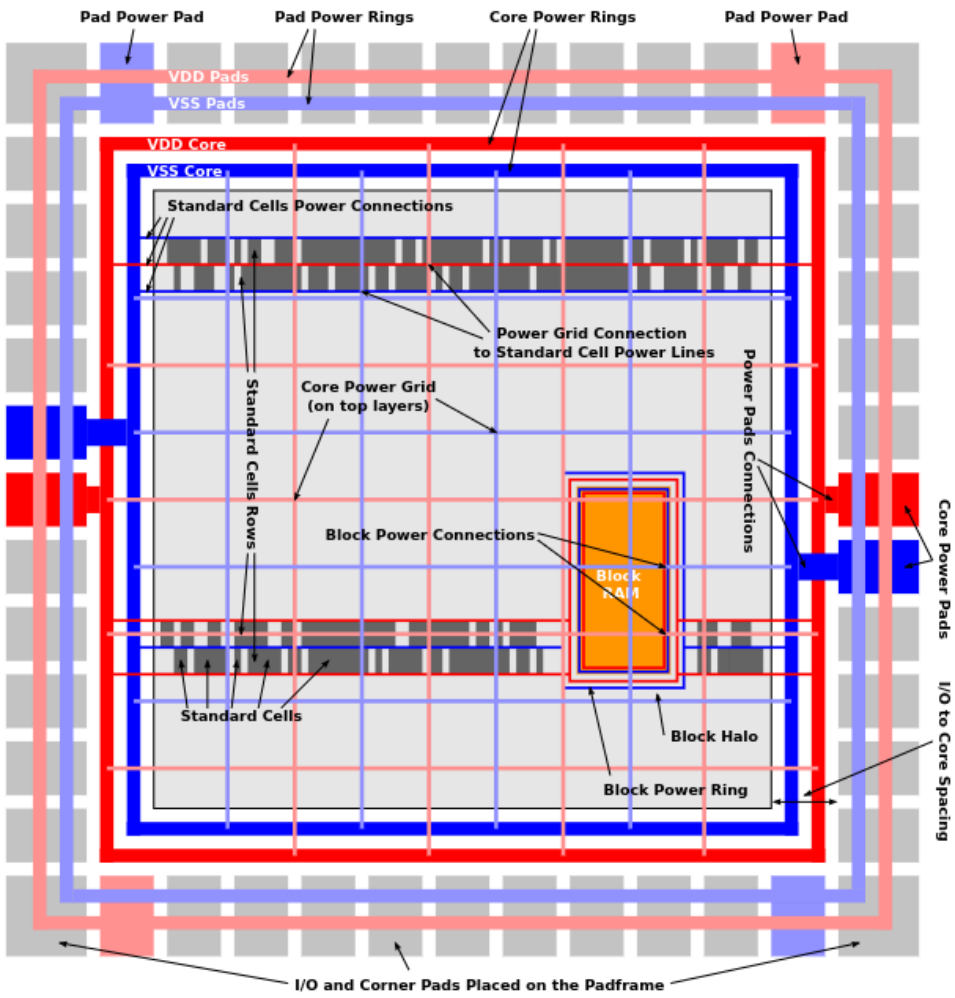

Figure 1: Floorplane example.

ezelioli -- Add better figure that clearly shows 1) sealring, 2) bondpads, 3) IOpads drivers, 4) power ring, 5) core area, with size annotations taken from real final chip size

Initialize Floorplan

Now that you know the structure of the floorplan, you can start the floorplanning design. The first step is to determine the die area and core area size.

Student Task 3

Go to the OpenROAD documentation `https://openroad.readthedocs.io/en/latest/` and look for the Floorplan Initialization.

- How many methods exist to set the floorplan dimensions in OpenROAD?
- Report an example of a command to initialize the floorplan for each method.
- What is the core space ?
- What is the aspect ratio ?

Now that you know which commands to use, you need to understand how to use them.

Student Task 4

Suppose that the manifacturing company is not giving you any constraint.

- Where do you find information about the core area ?
- Where do you find information about the pads dimensions ?
- How do you determine the die area? What should you account for when determining it?
- How do you determine the aspect ratio? When is it more convenient to have a rectangular shape and when a squared shape ?

Call an assistant to discuss about these questions before moving on.

So the problem you target to solve when initializing the floorplan is: *How small can my layout be so that I am still able to fulfill all specifications?*.

As a lower bound, you will need enough area to place all your I/O pads and standard cells. Ideally, in terms of area you will want to place standard cells without leaving extra space in between, completely filling out the core area. This is hardly ever possible because:

- The **number of interconnections** that can pass through a certain area is limited by the number of metal layers available, wire width and minimum spacing requirements. Depending on the interconnection overhead, the area above the cells may not be sufficient for routing.
- **Timing** is greatly affected by the placement of your cells. Placing them next to each other with no space in between does not leave the tool any flexibility in placing cells. This in turn reduces the optimization options of the tool, like the ability to cluster cells that are closely interconnected.
- All designs require **power routing** for operation. Some wires of the power connection limit where the cells can be placed, or restrict signal routing which in turn increases the area requirement.
- The majority of designs require a **clock tree** to function. This clock tree is added during the back-end design. This requires additional area for the buffers used in the clock tree. Furthermore, the clock tree synthesis algorithm can produce better results if it has more freedom to place its buffers.
- **Macro-cells**, like the RAM in our example, usually require some extra space along the edges so that they can be properly integrated next to the standard cells.
- Designs that have a **high switching activity** require a lot of current for a short time which is called a surge. The power distribution network may need additional decoupling capacitors to store some charge that can provide some of the current of the standard cells during such a surge. Additional space for these decoupling cells may be required during placement.

As a consequence, the standard cell rows can not be filled completely with standard cells. There needs to remain some free space between the cells. Utilization indicates to what percentage the standard cell rows are filled. Usually, it is not possible to predict whether all requirements can be fulfilled with a certain utilization. Both placement and routing are NP complete problems. Without going through a complete routing and placement, it is not possible to know. You will have to try and find out, which is the main reason why **back-end design is an iterative process**.

Student Task 5

Apply what you learned to Croc.

- Define the dimensions of the chip with the following commands:

```
OpenROAD > set chipW            1916.0
OpenROAD > set chipH            1916.0
```

- Following the comments in the provided script, define the pad depth, bond with the folloiwng commands:

```
OpenROAD > set padD            180.0
OpenROAD > set padBond          70.0
```

- Define the reserved space for the powerRings with the following commands:

```
OpenROAD > set powerRing        80;
* Finally, define the core margin:
OpenROAD > set coreMargin [expr {$padD + $padBond + $powerRing}];
```

- Initialize the floorplan with the following command:

```
OpenROAD > initialize_floorplan -die_area "0 0 $chipW $chipH" -core_area "$coreMargin $coreMargin [expr
$chipW-$coreMargin] [expr $chipH-$coreMargin]" -site "CoreSite"
```

After initializing the floorplan, you should see a message like the one below. Note that the actual core area is not exactly the one specified with the `initialize_floorplan` command, because it must be aligned to the core site grid.

```
Core area lower left (330.000, 330.000) snapped to (330.120, 332.640).
```

At this point you can visualize the shape of your chip on the OpenROAD GUI, which means that you have successfully initialized the floorplan. However, you might end up realizing in the following steps that these constraints are not suitable for your chip. Instead of rewriting these commands every time, you can save them in a TCL script.

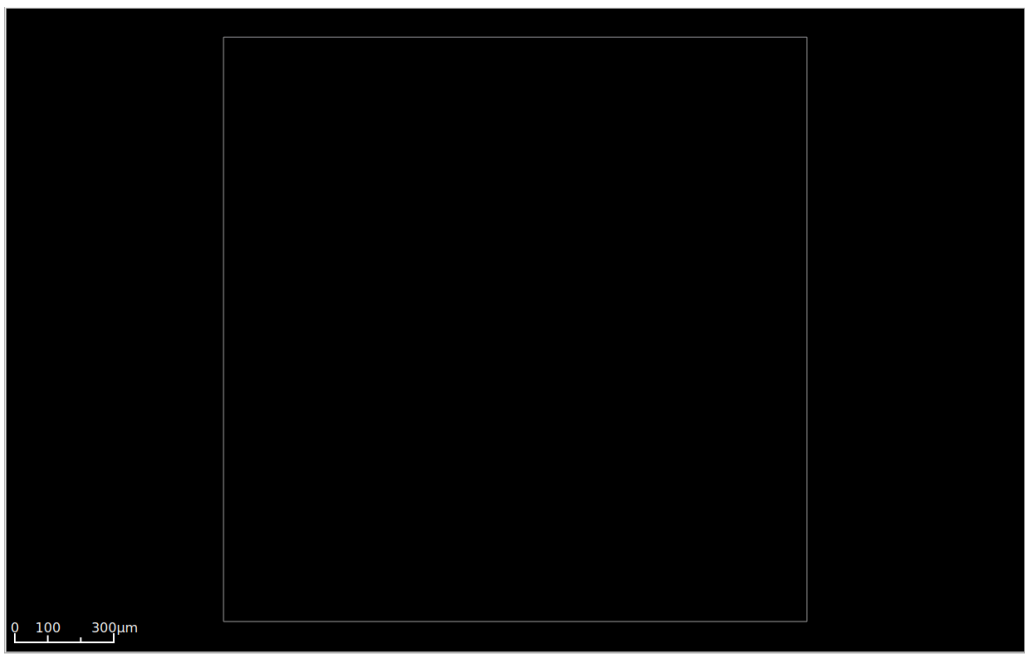

Figure 2: Shape of the Croc chip displayed on the OpenROAD GUI

ezelioli -- This figure should actually show both the die and core areas.

IO Placement

Microchips require specialized circuitry for electrostatic discharge (ESD) protection and strong buffers to drive large off-chip loads. I/O drivers do include all these circuits and also include **bonding pads** that enable physical connection between the microchip and the package. I/O drivers are basically specialized buffers, much larger than regular standard cells, and are designed to be placed around the core.

In addition to the core design and the I/O drivers, the pad frame will also contain pads for power and ground supply for both the core and the I/O drivers, as well as **filler cells** and **corner cells** to connect the edges of the pad frame. There are many different ways in which we can obtain a netlist describing the pad frame. The necessary information about the IO cells (physical size, pin names, driving strength and so on) are found in the PDK.

In this exercise, the top module has already been written for you.

Student Task 6

Open `rtl/croc_chip.sv` in rtl directory.

- Which modules instantiate power pads?
- Which modules instantiate input signals pads?
- How many power pads are defined?
- How many pads are instantiated overall?

Go to `technology` directory.

- In which files do you find geometrical information on the pads?
- Report the size, the symmetry and the site of the input signals pads.

The pad placement is very important as it directly determines the PCB layout.
The pad placement affects the routing (in both IC and PCB) and the signal integrity, which impacts both on performance and energy efficiency.
Moreover, the power supply and the thermal management depend on the power pads placement.
Finally, the pad placement needs to take into account the PCB mechanical constraints.

Student Task 7

As IC designer, you have to decide **which edges** to use, how to **distribute the pads** for every edge and **define the pin placement**.

- Is it convenient to place the pads on a single edge? If yes, when?
- What is the main drawback of asymmetrical pads placement? Would you recommend it?
- What is the main advantage of using all edges ?
- How would you place power pads and why? How many edges would you use for the croc chip?
- Would you place high frequency signals (clock) close to sensitive signals (power supply or analog signals)? Why?
- How would you place signals connected to the same submodules (JTAG)? Closely or not? Why?

- Analyze the pad placement shown in the figure below. Is this a good design?

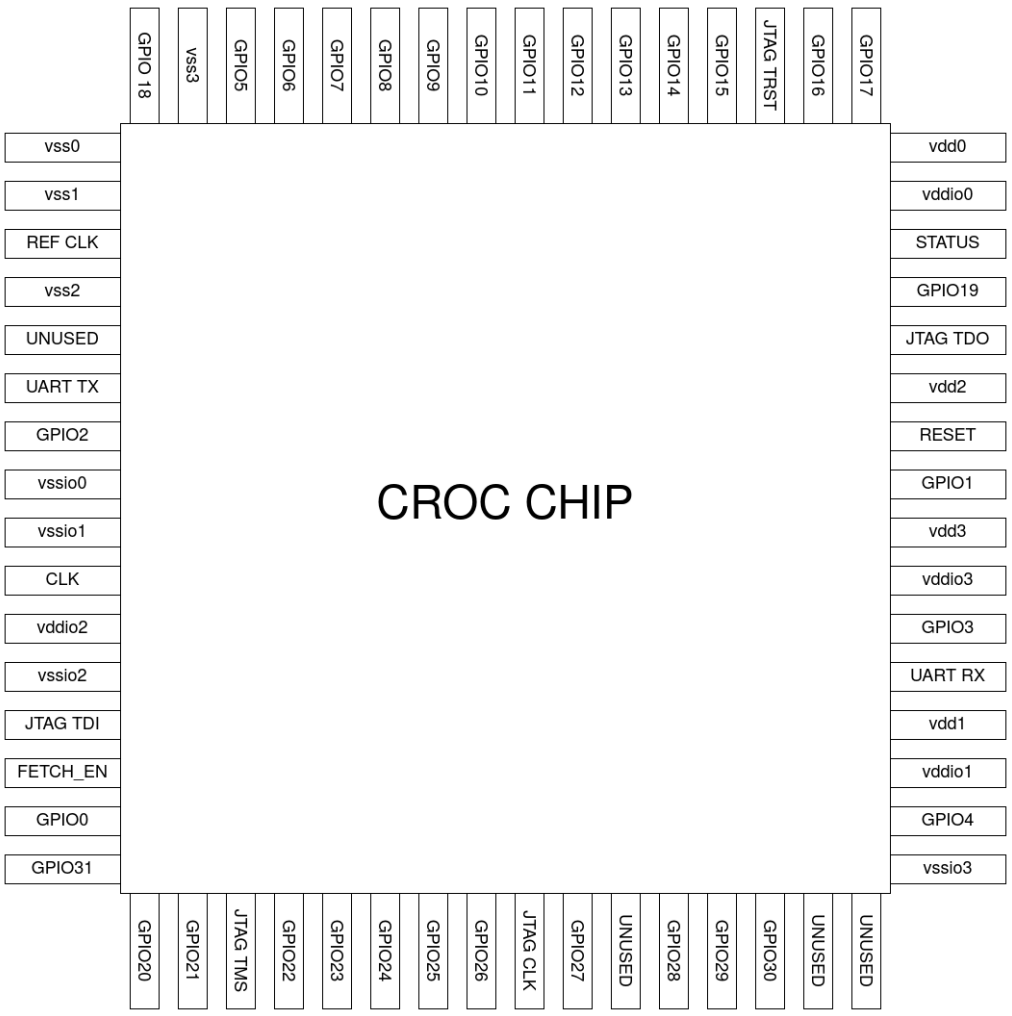

Figure 3: Example of the pin placement drawing for the Croc chip

- Open `rtl/croc_chip.sv` to read the pads name that need to be placed in your system.
- Sketch a drawing of your chip where you place the pads. This will help you in the next section.

Call an assistant to discuss about these questions before moving on.

Now we also need to provide OpenROAD with the information about where on the die each pad should be placed. OpenROAD allows you to precisely specify the location of the IOs with a few commands.

Student Task 8

In this exercise, you are going to modify a TCL script to execute the pin placement commands.
You find the template in `src/padring.tcl`.

Now you have to instruct the tool to define sites for the pads to be placed into.

- Go to the OpenROAD documentation `https://openroad.readthedocs.io/en/latest/` and look for Chip-level connections.
- Scroll down the page until you find the command to **make an IO site**.
- Read the description of all the possible options you can specify.
- Copy the command into the `src/padring.tcl` file.
- Specify as `horizontal_site`, `vertical_site`, `corner_site`, the name of the site that you found in the LEF file for the IO pads.
- We need to specify an `offset` equal to the `padBond`, so write `$padBond` .

This command takes a site (a concept from the LEF files) and uses it to build rows and columns of them.

Let's place the pads.

- Go to the OpenROAD documentation `https://openroad.readthedocs.io/en/latest/` and look for Chip-level connections.
- Scroll down the page until you find the command to **placing pads manually**.
- Read the description of all the possible options you can specify.
- The values of padW and padD have been found in the task 6 and already defined in task 5 (padW is the smallest value).
- The values of chipH and chipW have been specified in the task 5.
- Then, you need to specify the number of pads for each edge.
- Finally, you need to place the pads for each edge, specifying the name for each pad.
- Follow the provided template in `src/padring.tcl` and modify it accordingly to your pin placement decision in task 7.

At this point, the pads are placed. What is still needed to specify to the tool are the corner cells, the filler cells and the bonding pads.

- Go to the OpenROAD documentation `https://openroad.readthedocs.io/en/latest/` and look for Chip-level connections.
- Scroll down the page until you find the command to **placing corner cells**.
- Read the description of all the possible options you can specify.
- The master has already been saved in a variable, so you can specify it as `$iocorner`.
- Copy the command, save it into `src/padring.tcl` and modify it accordingly.

- Go to the OpenROAD documentation `https://openroad.readthedocs.io/en/latest/` and look for Chip-level connections.
- Scroll down the page until you find the command to **place filler cells**.
- Read the description of all the possible options you can specify.
- The masters have already been saved in a variable, so you can specify them as `{*}$iofill`.
- Copy the commands, save them into `src/padring.tcl` and modify them accordingly.

- Go to the OpenROAD documentation `https://openroad.readthedocs.io/en/latest/` and look for Chip-level connections.
- Scroll down the page until you find the command to **connect ring signals**.
- Copy the command, save it into `src/padring.tcl`.

- Go to the OpenROAD documentation `https://openroad.readthedocs.io/en/latest/` and look for Chip-level connections.
- Scroll down the page until you find the command to **place bonding pads**.
- Read the description of all the possible options you can specify.
- Copy `place_bondpad -bond $bondPadCell -offset {5.0 -70.0} pad_*` and save it into `src/padring.tcl`.

- Go to the OpenROAD documentation `https://openroad.readthedocs.io/en/latest/` and look for Chip-level connections.
- Scroll down the page until you find the command to **remove the IO rows**.
- Copy the command and save it into `src/padring.tcl`.

At this point, you should have saved all the commands that you use in the `src/padring.tcl` file.
Then, you can run the following command to read the tcl file.

```
OpenROAD >  source src/padring.tcl
```

Backend is an iterative process, so you might understand later in the flow your decisions were not the best. Then, if you are not satisfied by your pin placement, modify the `src/padring.tcl` file and run it again on the GUI.

Macro Placement

The next step is the macro placement.

When placing a macro-cell, you have to decide **where to locate it** and **how it is oriented**.
In order to make such decisions, you have to remember that a macro will completely block some metal layers and this can easily create congested areas.
Furthermore, according to where the power and signal pins of the block are located and which metal layer they are on, you might prefer them facing the core and not the I/O pads (or viceversa).
Finally, a macro cell requires a halo to prevent standard cells from being placed too close and a power ring, impacting on the routing area.

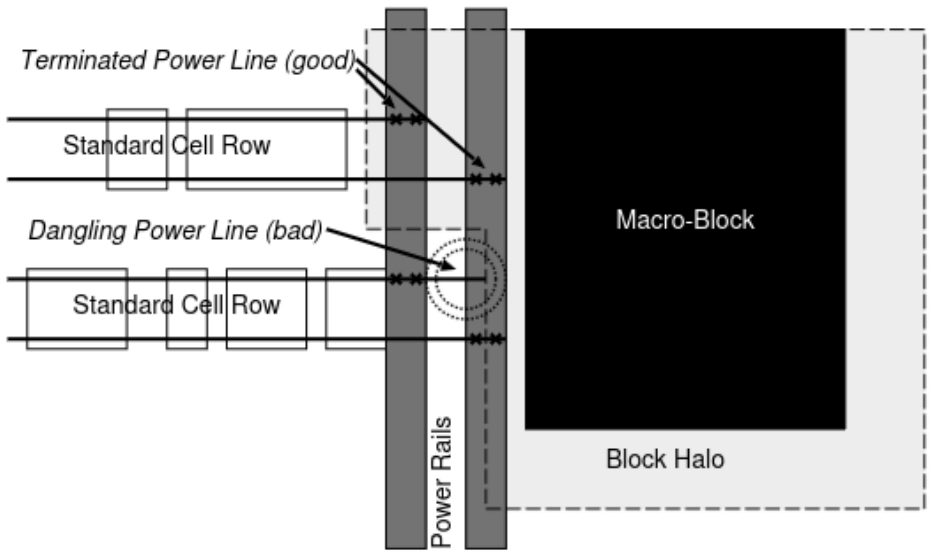

Figure 4: Halo conflicting with power rails.

Student Task 9

Let's try to understand which decisions are preferrable in some cases.
Hint: represent each case on a paper to facilitate your understanding!

- Which are the advantages and drawbacks of placing a macro in the middle of the core area?
- Which are the advantages and drawbacks of placing a macro close to one edge or a corner?
- If you have multiple macros, what is the advantage of placing them close to each other?
- If you have multiple macros, why might you prefer leaving some room among them?

Call an assistant to discuss about these questions before moving on.

At this point, let's place the SRAM macro-cell. The RAM macro will completely block the lower four metal layers (M1 - M4). The remaining metal layers will be available for routing over the RAM macro-cell but TM1 and TM2 are not really meant to have a lot of signal routing on them, so when you place macros close together you can easily create congested areas.

> *ezelioli -- Elaborate on the fact that with this metal stack SRAM macros placement is quite critical since we only have one TM layer left over the macro. We therefore cannot place two SRAMs on top of each other because we can't route all signals on TM1 layer, and routing around requires leaving a lot of space between them. It is therefore best to place them next to each other. SRAM macros also cannot be rotated.*

Student Task 10

As specified in `rtl/croc_pkg.sv` (`https://github.com/pulp-platform/croc/blob/d861c75454af8b8708704ec535bae53f10d03fb9/rtl/croc_pkg.sv#L63`), there are 2 SRAM banks to be placed.

- Go to the OpenROAD documentation `https://openroad.readthedocs.io/en/latest/` and look for Macro Placement.
- Read the description of all the possible options you can specify.

Instead of using the OpenROAD built in `place_macro`, you will use a helper function from `openroad/scripts/floorplan_util.tcl` called `placeInstance` which supports additional orientations.
The command syntax is `placeInstance name X Y orientation`.
The possible orientations are listed here `https://github.com/The-OpenROAD-Project/OpenROAD/blob/12d2d36d02eccae67065acac41c98b03e32e0715/src/odb/include/odb/dbTypes.h#L58`

- The helper function has been already sourced in the startup script.
- Consequently, you need to **specify the name of the instances** that you find in the RTL code. We provided a script with some instances anme already defined for you: `src/instances.tcl`. Add the instance name of the two memory macros as below:

```
OpenROAD > set sram {\[0\].i_sram/}
OpenROAD > set bank0_sram0 $SRAM$sram$SRAM_512x32
OpenROAD > set sram {\[1\].i_sram/}
OpenROAD > set bank1_sram0 $SRAM$sram$SRAM_512x32
```

- To compute the location, you firstly need to **get the core dimensions**.

```
OpenROAD > set coreArea      [ord::get_core_area]
OpenROAD > set core_leftX    [lindex $coreArea 0]
OpenROAD > set core_bottomY  [lindex $coreArea 1]
OpenROAD > set core_rightX   [lindex $coreArea 2]
OpenROAD > set core_topY     [lindex $coreArea 3]
```

- Then, you can **define some reference locations**, which can be used in the computation of the macro placement.

```
OpenROAD > set floorPaddingX      ...
OpenROAD > set floorPaddingY      ...
OpenROAD > set floor_leftX       [expr $core_leftX + $floorPaddingX]
OpenROAD > set floor_bottomY     [expr $core_bottomY + $floorPaddingY]
OpenROAD > set floor_rightX      [expr $core_rightX - $floorPaddingX]
OpenROAD > set floor_topY        [expr $core_topY - $floorPaddingY]
OpenROAD > set floor_midpointX   [expr $floor_leftX + ($floor_rightX - $floor_leftX)/2]
OpenROAD > set floor_midpointY   [expr $floor_bottomY + ($floor_topY - $floor_bottomY)/2]
```

The floorPadding variables are used to define an area where not to place macros around the edges of the core.
You can specify whichever value you want, for example `20.0`.
But remember: if you are not satisfied with the value you specified, you can change it! Backend is an iterative process.

- Finally, you need to **get the macros dimensions**.

```
OpenROAD > set RamMaster256x64   [[ord::get_db] findMaster "RM_IHPSG13_1P_256x64_c2_bm_bist"]
OpenROAD > set RamSize256x64_W   [ord::dbu_to_microns [$RamMaster256x64 getWidth]]
OpenROAD > set RamSize256x64_H   [ord::dbu_to_microns [$RamMaster256x64 getHeight]]
```

- Instantiate the SRAM macro the following command: `placeInstance name X Y orientation`.
- Decide the location (X,Y) and the orientation (try different values, backend is an iterative process!).
- Once the macros are placed, you need to **specify the halo** with the following command.

```
OpenROAD > cut_rows -halo_width_x 2 -halo_width_y 1
```

Power Planning

When designing the power placement, you target to distribute power to each part of the design equally.

When routing the power signals, your concerns are mainly the electromigration and the IR drops. The electromigration is a phenomenon due to the movement of atoms which can cause shorts, open circuits, performance degradation or device failure. It is proportional to temperature and current density, so wider wires better mitigate the electromigration. The IR drops instead, is a voltage drop due to the resistance of the wires.

A chip might need multiple power domains: the power domains and the signal crossing from one domain to another are specified in the Power Intent. The power intent is written either in the Unified Power Format (UPF) or in the Common Power Format (CPF).

The VDD and VSS signals are global connections because they are connected to all cells. While upper metal layers are more suitable for distributing the power across the chip, standard cells require power signals at M1. Consequently, the power distribution follows the subsequent hierarchy:

- **rings** carry VDD and VSS around the chip (you can think at them as the beltways).
- **stripes** carry VDD and VSS across the chip (you can think at them as the main avenues).
- **rails** carry VDD and VSS to the standard cells (you can think at them as the streets).

When designing the power placement, you have to decide on which layers to place rings, stripes and rails.
And for each layer, the **pitch** and the **spacing**.
The pitch is the distance between two adjacent VDD or two adjacent VSS in a metal layer. The spacing is the minimum distance between adjacent VDD and VSS. The vertical connections between metal layers are called **vias**.

The spacing and the pitch impact on the power integrity and on the routing congestion. Moreover, some electromagnetic effects (crosstalk, coupling capacitors) might show up in bad power placement, degrading the overall performance.

Student Task 11

- Which blocks need a power ring?
- Which metal layers are more suitable for reducing electromigration and IR drop?

Hint: for the following questions, represent each case on a paper to facilitate your understanding!

- What is the advantage of having a small pitch? What is the drawback?
- What is the advantage of having a large pitch? What is the drawback?
- What is the advantage of having a small spacing? What is the drawback?
- What is the advantage of having a large spacing? What is the drawback?

Let's see now how to define a power placement with OpenRoad.

Student Task 12

- Go to the OpenROAD documentation `https://openroad.readthedocs.io/en/latest/` and look for PDN Generation.
- Suppose that the UPF file is called `croc.upf`. Write an example of the command to read such UPF file.

In this case, you do not use any UPF file, therefore OpenROAD complains that it cannot find it.

The first step in the power planning is to **define the global connections**.

- Go to the OpenROAD documentation `https://openroad.readthedocs.io/en/latest/` and look for OpenROAD APIs.
- Scroll down the page until you find the command to **add global connection**.
- Read the description of all the possible options you can specify.

Note that the `-pin_pattern` can be found in the LEF.

- For sake of simplicity, the commands to add the global connections are given here to you.

```
OpenROAD > source scripts/power_connect.tcl
```

Once the supply pins of the standard cells, pads, macros and bondpads are known to the tool to be global connection, they just need to be connected.

- Go to the OpenROAD documentation `https://openroad.readthedocs.io/en/latest/` and look for OpenROAD APIs.
- Scroll down the page until you find the command to **perform global connection**.
- Run the command on the GUI terminal.

At this point, you can define the power domains.

- Go to the OpenROAD documentation `https://openroad.readthedocs.io/en/latest/` and look for PDN Generation.
- Scroll down the page until you find the command to **define a voltage domain**.
- Read the description of all the possible options you can specify.
- The name of the voltage domain you create is `CORE`, while the power and ground names are `VDD` and `VSS`.

Consequently, the command you need to run is the following

```
OpenROAD > set_voltage_domain -name {CORE} -power {VDD} -ground {VSS}
```

Student Task 13

Then, you can start building up the power grid.
We provided the script `scripts/power_grid.tcl` with some some variables you will use.

- Go to the OpenROAD documentation `https://openroad.readthedocs.io/en/latest/` and look for PDN Generation.
- Scroll down the page until you find the command to **define a power grid**.
- Read the description of all the possible options you can specify.

As you can read, there are several options to **define a power grid**, and **macros require a separate power grid**.
Therefore, you need to define 2 power grids: one for the core, and one for the macros.

The core power grid does not have special requirements, so the command usage will be similar to the Method 1 example. You just need to assign a voltage domain and a unique name to the power grid for the core. The command will be then `define_pdn_grid -name {...} -voltage_domain {...}`.

- Name the core power gride `core_grid` and associate to it the voltage domain that you previously created.

Once you have written the complete command, you can run it.

Instead, the macro power grid command usage will be similar to the Method 2 example.

- You need to specify to which cells you want to associate the macro power grid. In this example, the cells name is saved in `$macro`.
- You need to assign a name to the this power grid, for example `sram_256x64_grid`.
- You need to specify an orientation for a macro power grid, for example `R0 R180 MY MX`.
- You need to specify the command to place the power grid over the entire macro, to associate it to a voltage domains and eventually the halo.
- In the end, the command will be:

```
OpenROAD >  define_pdn_grid -macro \
   -cells $macro \
   -name sram_256x64_grid \
   -orient "R0 R180 MY MX" \
   -grid_over_boundary \
   -voltage_domains {CORE} \
   -halo {1 1}
```

Student Task 14

Now you can focus on the core grid first.

- Go to the OpenROAD documentation `https://openroad.readthedocs.io/en/latest/` and look for PDN Generation.
- Scroll down the page until you find the command to **add a PDN ring**.
- Read the description of all the possible options you can specify.
- You need to specify which metal layers to use (one for vertical wires and one for horizontal wires), the wire width and spacing, and connect them.
- As this grid is the top-level one and will be connected to the pads, you need to specify that and also tell the pad pin layers.
- Eventually, you can also specify an pad offset.
- In the end, the command will be:

```
OpenROAD >  add_pdn_ring -grid {core_grid} \
  -layer        {TopMetal1 Metal4} \
  -widths       "$pgcrWidth $pgcrWidth" \
  -spacings     "$pgcrSpacing $pgcrSpacing" \
  -core_offsets "$pgcrOffset $pgcrOffset" \
  -add_connect                          \
  -connect_to_pads
```

Next we add the standard cell power rails on M1, this is where the power should go in the end.

- Go to the OpenROAD documentation `https://openroad.readthedocs.io/en/latest/` and look for PDN Generation.
- Scroll down the page until you find the command to **add a PDN stripe**.
- Read the description of all the possible options you can specify.
- You need to specify the metal layer, the width of the stripe and the offset.
- You specify that this stripe is part of the standard cells rails with `-followpins`.
- You specify that you want to extend the stripes up to the power grid ring with `-extend_to_core_ring`.
- In the end, the command will be:

```
OpenROAD >  add_pdn_stripe -grid {core_grid} \
  -layer {Metal1} \
  -width {0.32} \
  -offset {0} \
  -followpins \
  -extend_to_core_ring
```

Finally, you can **add vias to connect** the standard cells layer to the pad layers.

- Go to the OpenROAD documentation `https://openroad.readthedocs.io/en/latest/` and look for PDN Generation.
- Scroll down the page until you find the command to **add a PDN connect**.
- Read the description of all the possible options you can specify.
- Write the command to connect the standard cells layer to the pad layers.

Currently, the resistance (and with that the voltage drop) is dominated by the relatively thin but long Metal1 stripes.
So let's support the power rails by regularly injecting power from a path with less resistance.

- Design and run a power stripe to reduce the resistance. Read the options to understand how to use them.

`add_pdn_stripe -grid {core_grid} -layer ... -width ... -pitch ... -spacing ... -offset ... -extend_to_core_ring -snap_to_grid -number_of_straps 7`

Student Task 15

Now you have to do something similar for the macro power grid.

- For this purpose we have provided the tcl process `sram_power`.
- To use it add the following command:

```
OpenROAD >  sram_power "sram_256x64"  "RM_IHPSG13_1P_256x64_c2_bm_bist"
```

- What is the ring width?
- What is the spacing?

Student Task 16

Finally, you have to generate the PDN.

- Go to the OpenROAD documentation `https://openroad.readthedocs.io/en/latest/` and look for PDN Generation.
- Scroll down the page until you find the command to **build a power grid**.
- Read the description of all the possible options you can specify.
- You are interested in knowing which are the failed vias only, so the command is

```
OpenROAD > pdngen -failed_via_report ${report_dir}/01_${proj_name}_pdngen.rpt
```

Summary

You are done with this Exercise.

Discuss your experience with assistants and collegues or continue to explore the floorplan on your own.

The VLSI pages are part of the open source VLSI design course offered by the Integrated Systems Laboratory (https://iis.ee.ethz.ch) of ETH Zürich (https://www.ethz.ch), by Luca Benini (https://ee.ethz.ch/the-department/people-a-z/person-detail.luca-benini.html) and Frank K. Gürkaynak (https://iis-people.ee.ethz.ch/~kgf/). See full list of contributors. Unless otherwise noted content available under Creative Commons BY (https://creativecommons.org/licenses/by/4.0/).